



PYTHON PROGRAMMING LANGUAGE

Elmurod Azodov
@the_elmurod

Python Programming Language

STAGE 0 — Programming & Python Foundations

Goal

Understand what Python is and how it executes before writing code.

Topics

- What Programming Is
- What Python Is (Language vs Implementation)
- CPython Overview
- Interpreter vs Compiler
- Python Execution Model
- Python Source Files
- Python Versions and Release Cycle
- Installing Python Correctly
- Python REPL
- Running Scripts
- Code Editors and IDEs
- Syntax Rules
- Indentation and Code Blocks
- Comments and Documentation Strings

STAGE 1 — Core Syntax & Expressions

Goal

Write correct Python statements and expressions.

Topics

- Variables and Assignment
 - Identifiers and Keywords
 - Built-in Data Types Overview
 - Dynamic Typing
 - Type Checking
 - Type Conversion
 - Input Handling
 - Output Formatting
 - Operators
 - Arithmetic
 - Comparison
 - Logical
 - Assignment
 - Bitwise
 - Membership
 - Identity
 - Expressions
 - Operator Precedence
-

STAGE 2 — Python Object Model & Primitive Types

Goal

Understand Python's object-based nature.

Topics

- Everything Is an Object

- Object Identity
 - Object Mutability
 - Integers
 - Floating-Point Numbers
 - Complex Numbers
 - Booleans
 - NoneType
 - Numeric Precision
 - Immutability Concepts
-

STAGE 3 — Strings & Text Processing

Goal

Master text handling, a core Python skill.

Topics

- String Creation
- Indexing
- Slicing
- String Immutability
- Common String Methods
- Searching and Replacing
- String Formatting
 - f-strings
 - format()
- Escape Characters
- Unicode

- Encoding and Decoding
 - Text Normalization
-

STAGE 4 — Control Flow & Logic

Goal

Control program execution accurately.

Topics

- Boolean Expressions
 - Truthy and Falsy Values
 - if Statements
 - elif Chains
 - else Blocks
 - Nested Conditions
 - match / case (Structural Pattern Matching)
-

STAGE 5 — Loops & Iteration

Goal

Repeat logic safely and efficiently.

Topics

- for Loops
- while Loops
- Loop Control Statements
 - break
 - continue
 - pass

- Nested Loops
 - Loop else Clause
 - Common Loop Patterns
 - Avoiding Infinite Loops
-

STAGE 6 — Core Data Structures

Goal

Store, access, and manipulate data correctly.

Topics

- Lists
 - Creation
 - Indexing
 - Slicing
 - Methods
 - Mutability
- Tuples
 - Packing
 - Unpacking
 - Immutability
- Sets
 - Uniqueness
 - Mathematical Operations
- Dictionaries
 - Keys and Values
 - Hashing Basics

- Dictionary Methods
 - Choosing Data Structures
-

STAGE 7 — Functions & Scope

Goal

Write reusable and maintainable code.

Topics

- Function Definitions
 - Calling Functions
 - Parameters vs Arguments
 - Return Values
 - Default Arguments
 - Keyword Arguments
 - Variable-Length Arguments
 - Scope Rules
 - Local vs Global Scope
 - LEGB Rule
 - Docstrings
 - Clean Function Design
-

STAGE 8 — Advanced Functions & Functional Concepts

Goal

Use Python's functional capabilities correctly.

Topics

- Functions as First-Class Objects

- Lambda Functions
 - map()
 - filter()
 - reduce()
 - Recursion
 - Closures
 - Decorators
 - Function Annotations
-

STAGE 9 — Error Handling & Robustness

Goal

Build fault-tolerant programs.

Topics

- Syntax Errors vs Runtime Errors
 - Exceptions
 - Built-in Exception Hierarchy
 - try / except
 - Multiple except Blocks
 - else Clause
 - finally Clause
 - Raising Exceptions
 - Custom Exceptions
 - Error Design Best Practices
-

STAGE 10 — Modules, Packages & Environments

Goal

Structure real Python projects professionally.

Topics

- Python Modules
- import Mechanics
- from ... import
- Import Aliases
- Python Packages
- **init.py**
- Module Search Path
- Virtual Environments
 - venv
- Package Installation
 - pip
- pyproject.toml
- Dependency Management
- poetry
- uv
- pipenv (Legacy Awareness)

STAGE 11 — File Handling & OS Interaction

Goal

Work with real files and system resources.

Topics

- File Paths

- Pathlib
 - File Modes
 - Reading Files
 - Writing Files
 - Appending Files
 - CSV Files
 - JSON Files
 - os Module
 - sys Module
 - Environment Variables
-

STAGE 12 — Object-Oriented Programming

Goal

Design structured and scalable programs.

Topics

- OOP Principles
- Classes and Objects
- Attributes
- Methods
- Constructors
- Instance Variables
- Class Variables
- Encapsulation
- Access Control
- Inheritance

- Method Overriding
 - Polymorphism
 - Abstraction
 - Abstract Base Classes
 - Magic Methods
 - Dataclasses
-

STAGE 13 — Advanced OOP & Design

Goal

Apply professional design practices.

Topics

- Multiple Inheritance
 - Method Resolution Order (MRO)
 - Composition vs Inheritance
 - Mixins
 - SOLID Principles
 - Design Patterns in Python
 - Anti-Patterns
-

STAGE 14 — Python Standard Library

Goal

Use built-in tools instead of reinventing the wheel.

Topics

- math
- random

- datetime
- collections
- itertools
- functools
- statistics
- enum
- typing
- copy
- pickle
- argparse
- logging

STAGE 15 — Iterators, Generators & Context Managers

Goal

Write memory-efficient and elegant code.

Topics

- Iterable vs Iterator
 - Iterator Protocol
 - Generators
 - yield
 - Generator Expressions
 - Context Managers
 - with Statement
 - Custom Context Managers
-

STAGE 16 — Memory, Performance & Internals

Goal

Understand Python at runtime level.

Topics

- Python Memory Model
 - Stack vs Heap
 - Reference Counting
 - Garbage Collection
 - Shallow vs Deep Copy
 - Mutability Pitfalls
 - Time Complexity
 - Space Complexity
 - Profiling
 - Optimization Techniques
-

STAGE 17 — Concurrency & Async Programming

Goal

Handle multiple tasks efficiently.

Topics

- Processes vs Threads
- Global Interpreter Lock (GIL)
- threading
- multiprocessing
- concurrent.futures
- Async Programming Model

- `asyncio`
 - Event Loop
 - Async Functions
 - Async Context Managers
-

STAGE 18 — Testing, Quality & Tooling

Goal

Ensure correctness and maintainability.

Topics

- Testing Principles
 - Unit Testing
 - `pytest`
 - Assertions
 - Fixtures
 - Mocking
 - Code Coverage
 - Linting
 - Formatting
 - PEP 8
 - Type Hinting
 - Static Type Checking (`mypy` / `pyright`)
 - `Ruff` / `Black`
-

STAGE 19 — Packaging, Distribution & Maintenance

Goal

Ship Python software properly.

Topics

- Project Layout Standards
 - Packaging Workflows
 - Versioning
 - Dependency Locking
 - Wheels
 - Publishing Packages
 - Backward Compatibility
 - Deprecation Strategy
-

STAGE 20 — Senior Python Engineering

Goal

Operate as a senior Python engineer.

Topics

- Writing Production-Grade Python
- Reading CPython Source (High Level)
- Performance Trade-offs
- Refactoring Strategies
- Debugging Complex Systems
- Code Reviews
- Documentation Standards
- Technical Decision Making
- Mentorship
- Long-Term Maintenance

uzb:

Python dasturlash tili

BOSQICH 0 — Dasturlash va Python asoslari

Maqsad

Python nima va qanday ishlashini tushunish.

Mavzular

- Dasturlash nima
 - Python nima (til va interpreter farqi)
 - CPython haqida qisqacha
 - Interpreter va kompilator farqi
 - Python dasturining ishlash jarayoni
 - Python fayllari va strukturalari
 - Python versiyalari va yangilanishlar
 - Python oʻrnatish
 - Python REPL bilan ishlash
 - Skriptni ishga tushirish
 - Kod muharrirlari va IDE
 - Sintaksis qoidalari
 - Indentatsiya va kod bloklari
 - Izohlar va hujjat yozish
-

BOSQICH 1 — Asosiy sintaksis va ifodalar

Maqsad

Toʻgʻri Python kodini yozishni oʻrganish.

Mavzular

- O‘zgaruvchilar va qiymat tayinlash
 - Identifikatorlar va kalit so‘zlar
 - Ma’lumot turlari (overview)
 - Dinamik tur
 - Tur tekshirish
 - Tur konvertatsiyasi
 - Foydalanuvchi kiritishi va chiqishi
 - Operatorlar:
 - Arifmetik
 - Solishtirish
 - Mantiqiy
 - Tayinlash
 - Bitwise
 - A’zolik (membership)
 - Identifikatorlar (identity)
 - Ifodalar va operator ustuvorligi
-

BOSQICH 2 — Python obykti va primitive turlar

Maqsad

Python obyekt modeli va asosiy turlarni tushunish.

Mavzular

- Hamma narsa obyekt
- Obyekt identifikatori
- Obyekt o‘zgarmasligi va o‘zgarmas turlar

- Integer, Float, Complex
 - Boolean
 - NoneType
 - Numerik aniqlik
-

BOSQICH 3 — Matnlar bilan ishlash

Maqsad

Matn bilan ishlashni puxta oʻrganish.

Mavzular

- String yaratish
 - Index va slicing
 - String oʻzgarmasligi
 - String metodlari
 - Qidirish va almashtirish
 - Formatlash:
 - f-string
 - format()
 - Escape belgilar
 - Unicode va kodlash
 - Matn normalizatsiyasi
-

BOSQICH 4 — Shartli operatorlar va mantiq

Maqsad

Dastur oqimini nazorat qilish.

Mavzular

- Boolean ifodalar
 - True/Falsy qiymatlar
 - if, elif, else
 - Nested shartlar
 - match / case (pattern matching)
-

BOSQICH 5 — Sikllar va iteratsiya

Maqsad

Kod takrorlash va boshqarish.

Mavzular

- for sikl
 - while sikl
 - break, continue, pass
 - Nested sikllar
 - sikl else bloki
 - Mashhur sikl naqshlari
 - Cheksiz sikllardan saqlanish
-

BOSQICH 6 — Asosiy ma'lumot tuzilmalari

Maqsad

Ma'lumotni saqlash va ishlash.

Mavzular

- List
 - yaratish, indekslash, slicing
 - metodlar, o'zgaruvchanligi

- Tuple
 - packing/unpacking
 - o'zgarmaslik
 - Set
 - noyob elementlar
 - matematik operatsiyalar
 - Dictionary
 - Keys/values
 - Hashing
 - metodlar
 - Ma'lumot tuzilmasini tanlash
-

BOSQICH 7 — Funksiyalar va scope

Maqsad

Qayta ishlatiladigan va toza kod yozish.

Mavzular

- Funksiya yaratish va chaqirish
- Parametrlar va argumentlar
- Return qiymatlar
- Default va keyword argumentlar
- Variable-length argumentlar
- Scope: Local/Global
- LEGB qoidasi
- Docstring
- Toza funksiya dizayni

BOSQICH 8 — Advanced functions & Functional concepts

Maqsad

Python funksional imkoniyatlarini puxta o‘rganish.

Mavzular

- Funksiyalar birinchi darajali obyekt
- Lambda funksiyalar
- map, filter, reduce
- Recursion
- Closures
- Decorators
- Function annotations

BOSQICH 9 — Exception va xatolarga chidamlilik

Maqsad

Xatolardan xavfsiz ishlash.

Mavzular

- Syntax vs Runtime errors
 - Exception turlari
 - try / except
 - Multiple except
 - else/finally
 - Custom exceptions
 - Raise
-

BOSQICH 10 — Modul, package va muhit

Maqsad

Katta loyihalar uchun strukturani tushunish.

Mavzular

- Modul va import
 - from ... import
 - Import alias
 - Package va **init.py**
 - Module path
 - Virtual environment: venv
 - Pip
 - pyproject.toml
 - Poetry, UV, pipenv (legacy)
-

BOSQICH 11 — File va OS bilan ishlash

Maqsad

Fayl va tizim bilan ishlash.

Mavzular

- File paths va pathlib
 - File modes
 - Read, write, append
 - CSV va JSON
 - os va sys
 - Environment variables
-

BOSQICH 12 — OOP asoslari

Maqsad

Katta va murakkab dasturlar yozish.

Mavzular

- Classes & Objects
 - Attributes, Methods
 - Constructor
 - Instance va class variables
 - Encapsulation, Access control
 - Inheritance, Overriding
 - Polymorphism
 - Abstraction & ABC
 - Magic methods, Dataclasses
-

BOSQICH 13 — Advanced OOP & Design

Maqsad

Professional dizayn va arxitekturani qo‘llash.

Mavzular

- Multiple inheritance & MRO
 - Composition vs Inheritance
 - Mixins
 - SOLID principles
 - Design patterns
 - Anti-patterns
-

BOSQICH 14 — Python Standard Library

Maqsad

Built-in vositalardan foydalanish.

Mavzular

- math, random, datetime
 - collections, itertools, functools
 - statistics, enum, typing
 - copy, pickle
 - argparse, logging
-

BOSQICH 15 — Iterators, generators & context managers

Maqsad

Xotira samaradorligi va elegant kod.

Mavzular

- Iterable vs iterator
 - Iterator protocol
 - Generators, yield
 - Generator expressions
 - Context manager & with
 - Custom context managers
-

BOSQICH 16 — Memory va performance

Maqsad

Python ichki ishlashini tushunish.

Mavzular

- Memory model
 - Stack vs Heap
 - Reference counting
 - Garbage collection
 - Shallow vs deep copy
 - Time & space complexity
 - Profiling va optimization
-

BOSQICH 17 — Concurrency & Async

Maqsad

Ko‘p vazifali dasturlar yozish.

Mavzular

- Threads vs Processes
 - GIL
 - threading, multiprocessing
 - concurrent.futures
 - Asyncio, Event loop
 - Async functions & context managers
-

BOSQICH 18 — Testing va code quality

Maqsad

Dastur to‘g‘ri va saqlash oson bo‘lishi.

Mavzular

- Unit testing
- pytest

- Assertions, Fixtures
 - Mocking
 - Code coverage
 - Linting & formatting
 - PEP8
 - Type hints
 - Static type check (mypy/pyright)
 - Black/Ruff
-

BOSQICH 19 — Packaging & distribution

Maqsad

Python dasturini tarqatish va boshqarish.

Mavzular

- Project layout
 - Packaging
 - Versioning
 - Dependency locking
 - Wheel
 - Publishing
 - Backward compatibility
 - Deprecation strategy
-

BOSQICH 20 — Senior Python engineer

Maqsad

Professional Python muhandisi bo‘lish.

Mavzular

- Production-grade Python
 - CPython source high-level
 - Performance trade-offs
 - Refactoring strategies
 - Debugging complex systems
 - Code review
 - Documentation
 - Technical decisions
 - Mentorship
 - Large codebase maintenance
-

◆ Backend va AI mosligi

- **Backend:** Async, OOP, packaging, testing, concurrency, logging → barcha Python core bor
 - **AI:** Data structures, memory efficiency, generators, performance, functional programming → barcha Python core bor
 - Frameworklar (FastAPI, Django, PyTorch, NumPy) → bu roadmap ustiga qoʻshiladi, ammo Python asoslari boʻlmasa ishlamaydi.
-

O'quv davri davomiyligi:

Bosqichlar va vaqt taxmini

Bosqich	%	Taxminiy o'rganish soati
0. Python asoslari	0–5%	4 soat (2 dars)
1. Sintaksis, operatorlar	5–10%	6 soat (3 dars)
2. Primitive turlar	10–15%	6 soat (3 dars)
3. Strings	15–20%	6 soat (3 dars)
4. If/else	20–25%	4 soat (2 dars)
5. Loops	25–30%	6 soat (3 dars)
6. Data structures	30–35%	8 soat (4 dars)
7. Functions	35–40%	8 soat (4 dars)
8. Advanced functions	40–45%	6 soat (3 dars)
9. Exception handling	45–50%	4 soat (2 dars)
10. Modules & env	50–55%	6 soat (3 dars)
11. File & OS	55–60%	6 soat (3 dars)
12. OOP asoslari	60–70%	12 soat (6 dars)
13. Advanced OOP	70–75%	6 soat (3 dars)
14. Std library	75–80%	6 soat (3 dars)
15. Iterators, generators	80–85%	6 soat (3 dars)
16. Memory & performance	85–87%	4 soat (2 dars)
17. Concurrency & Async	87–90%	8 soat (4 dars)

Bosqich	%	Taxminiy o‘rganish soati
18. Testing & code quality	90–95%	6 soat (3 dars)
19. Packaging & distribution	95–98%	4 soat (2 dars)
20. Senior engineer skills	98–100%	6 soat (3 dars)

Umumiy soat va darslar

- Umumiy vaqt: **120 soat**
- Har dars 2 soat, haftada 3 dars → **haftada 6 soat**
- $120 / 6 = 20$ **hafta**
- **20 hafta $\approx$ 5 oy**

va

- Agar haftada **3 kun, har dars 2 soat** bo‘lsa:
Python ni **taxminan 5 oyda** tugatish mumkin.
- Bu hisobda amaliy mashqlar va kichik testlar kiritilgan.